

Regression

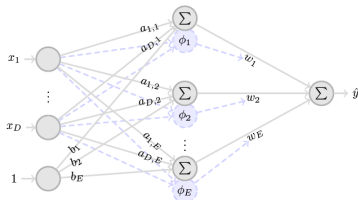
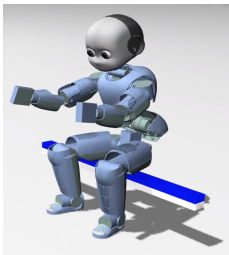
Olivier Sigaud

Sorbonne Université
<http://people.isir.upmc.fr/sigaud>

January 24, 2020



Motivations



Two main motivations for this class:

- ▶ Regression for robotics: approximating mechanical models
- ▶ Function approximation with deep neural networks, particularly deep RL
- ▶ More focus on the second



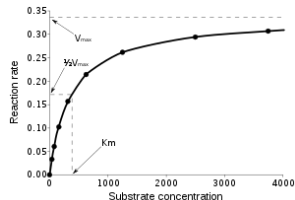
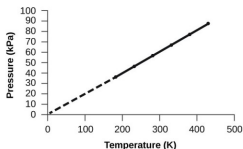
Sigaud, O., Salaün, C., & Padois, V. (2011) On-line regression algorithms for learning mechanical models of robots: a survey. *Robotics and Autonomous Systems*, 59(12):1115–1129

Outline

- ▶ Linear least squares
- ▶ Locally Weighted Regression
- ▶ Batch projection methods
- ▶ Gradient Descent methods
- ▶ Applications to RBFNs
- ▶ Applications to Neural Networks

Regression is function approximation

Temperature (°C)	Temperature (K)	Pressure (kPa)
-150	173	36.0
-100	223	46.4
-50	273	56.7
0	323	67.1
50	373	77.5
100	423	88.0



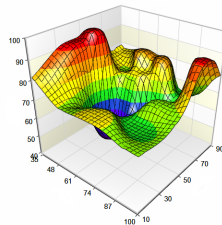
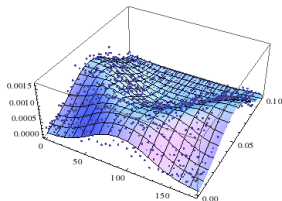
In regression:

- ▶ We have a set of related measurements
- ▶ We look for a **latent function** that relates the corresponding variables
- ▶ We choose a model over a class of (parametric) functions
- ▶ Here an example of **linear** (center) and **nonlinear** (right) regression

Nonlinear regression

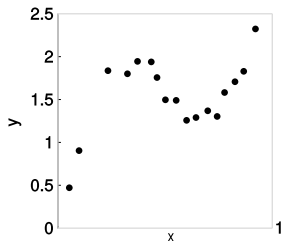
Sample Size(n)	λ	k	$BIAS$	CV
120	0 (without ridge)	30	6.12	6.01
100	1.15e-1	50	0.0030	0.0090
100	1.15e-5	20	2.52e-6	0.008
100	7.15e-5	20	1.57e-5	0.0085
100	1.15e-2	20	0.0023	0.0098
100	1.15e-3	20	2.50e-4	0.0083
100	1.15e-7	20	2.88e-8	0.009
100	8.15e-10	10	3.61e-6	0.0082
100	1.15e-3	10	0.0058	0.0096
100	7.15e-5	5	0.0179	0.0182
100	8.15e-10	5	0.0165	0.0177
120	1.15e-3	30	1.68e-5	0.0086
120	7.15e-5	30	0.0014	0.0093

Table 2 - The simulation results for two-dimensional dataset obtained by the proposed method.



- ▶ The function between variables can be arbitrarily complicated
- ▶ It can be multidimensional

Regression: basic process and notations

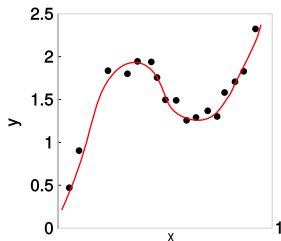


$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_{1,1} & \mathbf{x}_{1,2} & \cdots & \mathbf{x}_{1,D} \\ \mathbf{x}_{2,1} & \mathbf{x}_{2,2} & \cdots & \mathbf{x}_{2,D} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_{N,1} & \mathbf{x}_{N,2} & \cdots & \mathbf{x}_{N,D} \end{pmatrix} \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

(design matrix)

- ▶ The set of measurements is a **batch** of **datapoints**
- ▶ Input: N samples $\mathbf{x}_n \in \mathbb{R}^D$, $y_n \in \mathbb{R}$,
- ▶ Stored in $\mathbf{X} = [\mathbf{x}_1, \cdots, \mathbf{x}_N]$ (**design matrix**), $\mathbf{y} = [y_1, \cdots, y_N]$

Regression: latent function

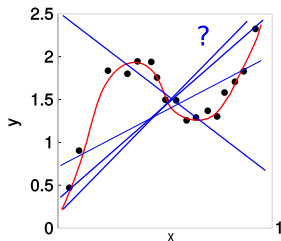


$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_{1,1} & \mathbf{x}_{1,2} & \cdots & \mathbf{x}_{1,D} \\ \mathbf{x}_{2,1} & \mathbf{x}_{2,2} & \cdots & \mathbf{x}_{2,D} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_{N,1} & \mathbf{x}_{N,2} & \cdots & \mathbf{x}_{N,D} \end{pmatrix} \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

(design matrix)

- ▶ The measurements correspond to an unknown function
- ▶ They can be subject to some noise
- ▶ We want to approximate the **latent function** (in red)

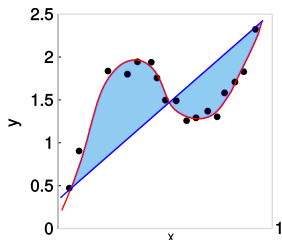
Regression: the output is a model



$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_{1,1} & \mathbf{x}_{1,2} & \cdots & \mathbf{x}_{1,D} \\ \mathbf{x}_{2,1} & \mathbf{x}_{2,2} & \cdots & \mathbf{x}_{2,D} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_{N,1} & \mathbf{x}_{N,2} & \cdots & \mathbf{x}_{N,D} \end{pmatrix} \quad \text{(design matrix)} \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

- ▶ We choose a parametric model over a class of functions
- ▶ Here we consider linear models (in blue)
- ▶ Output: a **model** $\hat{f}$ of the **latent function** f such that $\mathbf{y} \sim \hat{f}(\mathbf{X})$

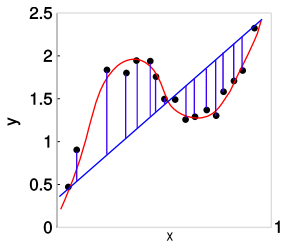
Regression: minimizing the expectation of the error



$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_{1,1} & \mathbf{x}_{1,2} & \cdots & \mathbf{x}_{1,D} \\ \mathbf{x}_{2,1} & \mathbf{x}_{2,2} & \cdots & \mathbf{x}_{2,D} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_{N,1} & \mathbf{x}_{N,2} & \cdots & \mathbf{x}_{N,D} \end{pmatrix} \quad \text{(design matrix)} \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

- ▶ We want to minimize some difference between f and $\hat{f}$
- ▶ An option would be the expectation of the error between f and $\hat{f}$ over the input space (light blue area)
- ▶ In practice, since we do not know f , we cannot compute this expectation

Regression: over a batch of samples

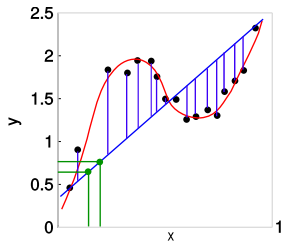


$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_{1,1} & \mathbf{x}_{1,2} & \cdots & \mathbf{x}_{1,D} \\ \mathbf{x}_{2,1} & \mathbf{x}_{2,2} & \cdots & \mathbf{x}_{2,D} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_{N,1} & \mathbf{x}_{N,2} & \cdots & \mathbf{x}_{N,D} \end{pmatrix} \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

(design matrix)

- ▶ We rather compute the error at the datapoints that we know
- ▶ We formalize this later

Regression: generalisation

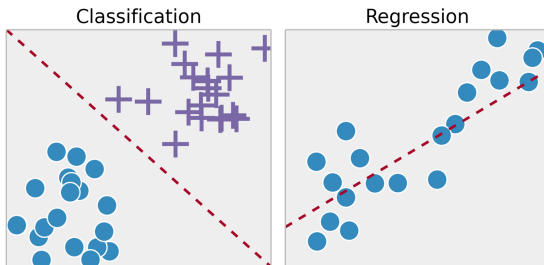


$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_{1,1} & \mathbf{x}_{1,2} & \cdots & \mathbf{x}_{1,D} \\ \mathbf{x}_{2,1} & \mathbf{x}_{2,2} & \cdots & \mathbf{x}_{2,D} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_{N,1} & \mathbf{x}_{N,2} & \cdots & \mathbf{x}_{N,D} \end{pmatrix} \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

(design matrix)

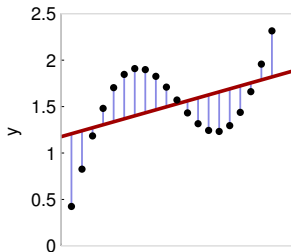
- If the error is low, the model will generalize well to unseen data (new $\mathbf{x}$)

Regression and classification



- ▶ Classification and regression are two forms of supervised learning
- ▶ In classification, the output space is a discrete set of classes, in regression it is continuous
- ▶ To perform classification, we usually rely on **labelled** databases (e.g. Imagenet)
- ▶ To perform regression, we rather rely on related measurements

Linear Least Squares



Black dots represent 20 training examples, and the thick red line is the learned model $\hat{f}(\mathbf{x})$. Vertical lines represent **residuals** $\|y - \hat{f}(\mathbf{x})\|$.

- ▶ We want to minimize the squared sum of the residuals
- ▶ In matrix form: $\min(\mathbf{y} - \hat{f}(\mathbf{X}))^2$.



Adrien Marie Legendre (1805) *Nouvelles méthodes pour la détermination des orbites des comètes*. F. Didot.



Carl Friedrich Gauss (1809) *Theoria motus corporum coelestium in sectionibus conicis solem ambientium*, volume 7. Perthes et Besser.

Offset trick

- ▶ In the linear case, we want $\mathbf{y} = \hat{f}(\mathbf{X}) = \boldsymbol{\theta}^\top \mathbf{X} + \mathbf{b}$
- ▶ We can remove the offset $\mathbf{b}$ by increasing $\mathbf{X}$ with a row of ones.

$$\hat{f}(\mathbf{X}) = \begin{pmatrix} \boldsymbol{\theta} \\ \mathbf{b} \end{pmatrix}^\top \begin{pmatrix} \mathbf{X} & \mathbf{1} \end{pmatrix}$$

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_{1,1} & \mathbf{x}_{1,2} & \cdots & \mathbf{x}_{1,D} & 1 \\ \mathbf{x}_{2,1} & \mathbf{x}_{2,2} & \cdots & \mathbf{x}_{2,D} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \mathbf{x}_{N,1} & \mathbf{x}_{N,2} & \cdots & \mathbf{x}_{N,D} & 1 \end{pmatrix}$$

- ▶ If we rewrite $\mathbf{y} = \hat{f}(\mathbf{X}) = \boldsymbol{\theta}^\top \mathbf{X}$, $\boldsymbol{\theta}$ is a vector of weights
- ▶ We minimize the residuals, thus

$$\boldsymbol{\theta}^* = \min_{\boldsymbol{\theta}} \underbrace{\|\mathbf{y} - \boldsymbol{\theta}^\top \mathbf{X}\|^2}_{L(\boldsymbol{\theta})}$$

Optimal linear model

$$L(\boldsymbol{\theta}) = \|\mathbf{y} - \boldsymbol{\theta}^T \mathbf{X}\|^2 \quad (1)$$

$$= (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^T (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) \quad (2)$$

$$= \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X}\boldsymbol{\theta} - (\mathbf{X}\boldsymbol{\theta})^T \mathbf{y} + (\mathbf{X}\boldsymbol{\theta})^T (\mathbf{X}\boldsymbol{\theta}) \quad (3)$$

$$= \mathbf{y}^T \mathbf{y} - 2(\mathbf{X}\boldsymbol{\theta})^T \mathbf{y} + (\mathbf{X}\boldsymbol{\theta})^T (\mathbf{X}\boldsymbol{\theta}) \quad (4)$$

At a minimum of a function, its derivative is null

$$\frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = 2(\boldsymbol{\theta} \mathbf{X}^T \mathbf{X} - \mathbf{X}^T \mathbf{y})$$

$$\frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = 0 \rightarrow \boldsymbol{\theta} \mathbf{X}^T \mathbf{X} = \mathbf{X}^T \mathbf{y}$$

Thus min reached where $\boldsymbol{\theta}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$

Regularized Least Squares

- ▶ Potential singularities in $\mathbf{X}^T \mathbf{X}$ can generate very large $\boldsymbol{\theta}^*$ weights
- ▶ Regularized Least Squares (Ridge Regression, RR): penalize large weights
- ▶ Optimize with lower weights (sacrifice optimality):



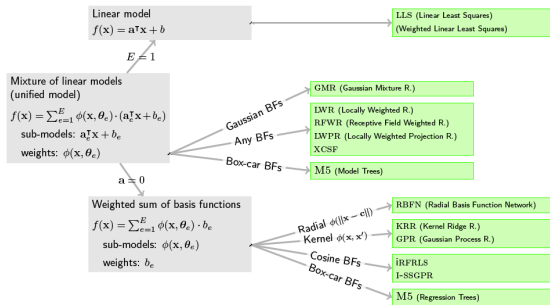
$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 + \frac{1}{2} \|\mathbf{y} - \mathbf{X}^T \boldsymbol{\theta}\|^2, \quad (5)$$

- ▶ Analytical solution:

$$\boldsymbol{\theta}^* = (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}. \quad (6)$$

- ▶ Tikhonov regularization = Ridge regression

Next slides: nonlinear regression

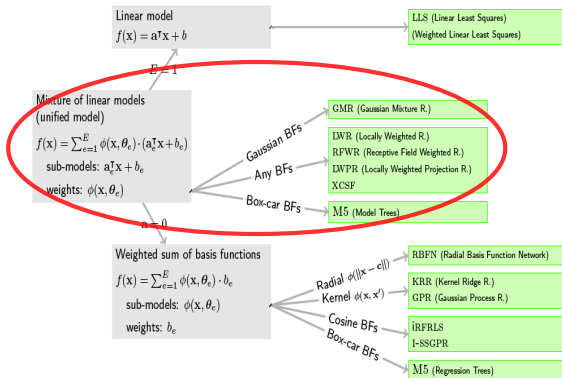


- ▶ Two different approaches:
 - ▶ Multiple local and weighted least square regressions (shown with LWR)
 - ▶ Projecting the input space into a feature space using non-linear basis functions (shown with RBFNs)
- ▶ We provide unifying views of algorithms from each family
- ▶ Then we highlight the similarity between both approaches



Stulp, F. and Sigaud, O. (2015). Many regression algorithms, one unified model: A review. *Neural Networks*, 69:60–79.

Reminder: Outline of methods



- Multiple local and weighted least square regressions (shown with LWR)



Stulp, F. and Sigaud, O. (2015) Many regression algorithms, one unified model: A review. *Neural Networks*, 69:60–79

Locally Weighted Regression

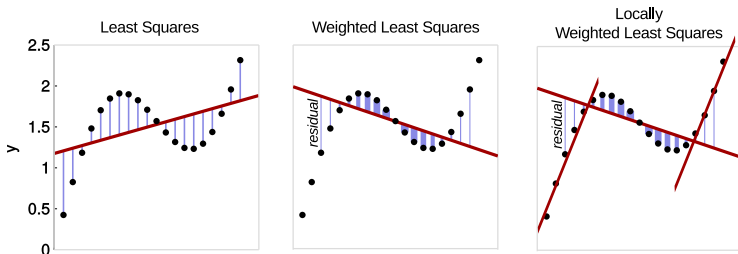


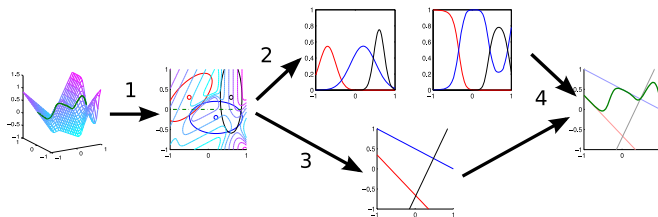
Figure: The thickness of the lines indicates the weights.

- ▶ General idea:
 - ▶ Split the function domain into linear parts
 - ▶ Give more weight to the centers
- ▶ Local linear models are tuned with Least Squares
- ▶ The importance of datapoints is represented by a Gaussian function



William S Cleveland and Susan J Devlin (1988) Locally weighted regression: an approach to regression analysis by local fitting.
Journal of the American statistical association, 83(403):596–610.

Locally Weighted Regression: Processes



1. Define the split into regions (**receptive fields**): by hand, data-driven, evolutionary
2. Determine relative importance of domains
3. Find linear models in the regions
4. Combine all linear models



Atkeson, C. (1991) Using locally weighted regression for robot learning. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, vol. 2, pp. 958–963

Combining linear models

- ▶ There are E features, or receptive fields (RF)
- ▶ Each RF is defined as a Gaussian $\phi(\mathbf{x}, \boldsymbol{\theta}_i) = e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i^{-1}(\mathbf{x}-\boldsymbol{\mu}_i)}$ with $\boldsymbol{\theta}_i = (\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$
- ▶ Each RF tunes a local linear model

$$\Psi_i(\mathbf{x}) = \mathbf{a}_i^T \mathbf{x} + b_i$$

- ▶ Gaussians tell you how much each RF contributes to the output

$$y = \frac{\sum_{i=1}^E \phi(\mathbf{x}, \boldsymbol{\theta}_i) \Psi_i(\mathbf{x})}{\sum_{i=1}^E \phi(\mathbf{x}, \boldsymbol{\theta}_i)}$$

Batch learning (1)

- ▶ Consider a batch of N $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{1 \leq i \leq N}$ data.



$$y = f(\mathbf{x}) = \frac{\sum_{i=1}^E \phi(\mathbf{x}, \boldsymbol{\theta}_i) \Psi_i(\mathbf{x})}{\sum_{i=1}^E \phi(\mathbf{x}, \boldsymbol{\theta}_i)}$$

with $\Psi_i(\mathbf{x}) = w(\mathbf{x})^\top \boldsymbol{\theta}_i$ and $w(\mathbf{x}) = (\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_d \ 1)^\top$.

- ▶ Each local model is computed using the following locally weighted error:

$$\begin{aligned} \epsilon_i(\boldsymbol{\theta}_i) &= \frac{1}{2N} \sum_{j=1}^N \phi(\mathbf{x}^{(j)}, \boldsymbol{\theta}_i) \left(y^{(j)} - \Psi_i(\mathbf{x}^{(j)}) \right)^2 \\ &= \frac{1}{2N} \sum_{j=1}^N \phi(\mathbf{x}^{(j)}, \boldsymbol{\theta}_i) \left(y^{(j)} - w(\mathbf{x}^{(j)})^\top \boldsymbol{\theta}_i \right)^2. \end{aligned}$$

Batch learning (2)

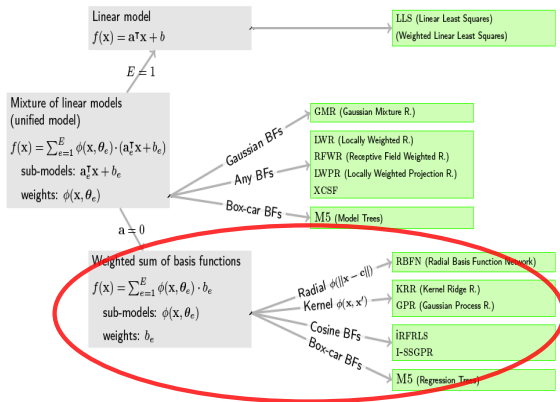
- ▶ As with the least squares method, we try to cancel out the gradient:

$$-\frac{1}{N} \sum_{j=1}^N \phi(\mathbf{x}^{(j)}, \boldsymbol{\theta}_i) w(\mathbf{x}^{(j)}) \left(y^{(j)} - w(\mathbf{x}^{(j)})^\top \boldsymbol{\theta}_i \right) = 0.$$

- ▶ Therefore, we pose $\boldsymbol{\theta}_i = A_i^\sharp b_i$, with:

$$A_i = \sum_{j=1}^N \phi(\mathbf{x}^{(j)}, \boldsymbol{\theta}_i) w(\mathbf{x}^{(j)}) w(\mathbf{x}^{(j)})^\top$$
$$b_i = \sum_{j=1}^N \phi(\mathbf{x}^{(j)}, \boldsymbol{\theta}_i) w(\mathbf{x}^{(j)}) y^{(j)}.$$

Reminder: Outline of methods

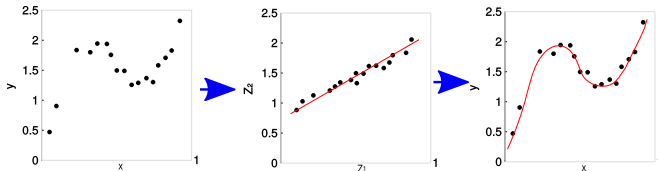


- Projecting the input space into a feature space using non-linear basis functions (shown with RBFNs)



Stulp, F. and Sigaud, O. (2015). Many regression algorithms, one unified model: A review. *Neural Networks*, 69:60–79.

Basis Function Networks: general idea

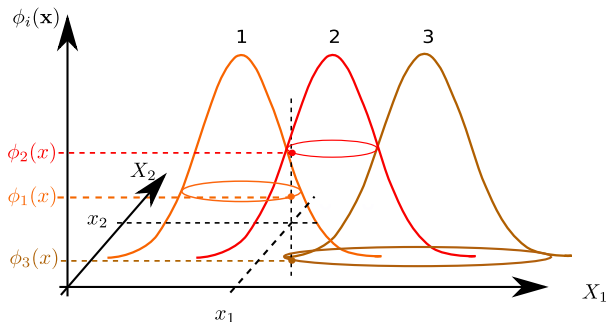


- ▶ With linear regression, we look for $\hat{f}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$
- ▶ This is not general enough for non-linear functions
- ▶ More general form: $\hat{f}(\mathbf{x}) = \sum_{e=0}^E w_e \cdot \phi_{\theta_e}(\mathbf{x})$ with $\phi_{\theta_0}(\mathbf{x}) = 1$
- ▶ This can be seen as projecting the input to a different space...
- ▶ ... where the latent function is linear



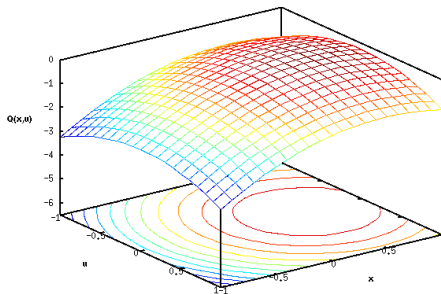
Bishop, C. M. (2007) *Pattern recognition and machine learning*. Springer Berlin/Heidelberg, Germany

Understanding projection



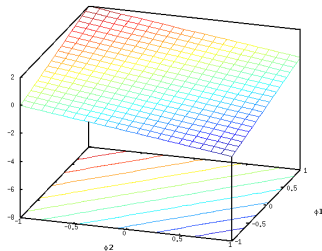
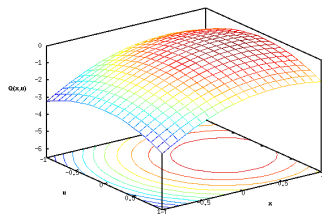
- The point $x = (x_1, x_2)$ is projected to $(\phi_1(x), \phi_2(x), \phi_3(x))$

Regression with features: example



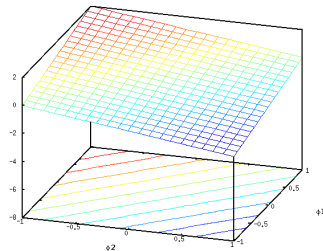
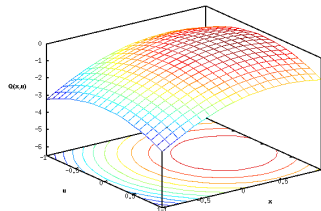
- ▶ The (unkown) function to be approximated is
$$f(x_1, x_2) = |x_1 - 2|^2 + 3 \cdot |x_2|^2 + 4$$
- ▶ We define features $\phi_i(x_1, x_2)$ over (x_1, x_2)
- ▶ We look for w such that $\hat{f}(x_1, x_2) = \sum_i w_i \phi_i(x_1, x_2)$

With poor features



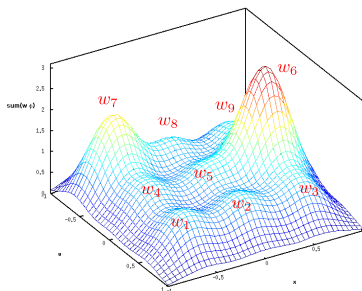
- ▶ If we take 3 feature functions $\phi_0(x_1, x_2) = 1$, $\phi_1(x_1, x_2) = x_1$ and $\phi_2(x_1, x_2) = x_2$
- ▶ We cannot do better than $\hat{f}(x_1, x_2) = w_1x_1 + w_2x_2 + c$
- ▶ Very poor linear approximation

With good features



- ▶ If we take $\phi_0(x_1, x_2) = 1$, $\phi_1(x_1, x_2) = |x_1 - 2|^2$ and $\phi_2(x_1, x_2) = |x_2|^2$
- ▶ Then $\hat{f}(x_1, x_2) = w_0 + w_1|x_1 - 2|^2 + w_2|x_2|^2$
- ▶ If we take $w_0 = 4$, $w_1 = 1$ and $w_2 = 3$, we get exactly $\hat{f}(x_1, x_2) = |x_1 - 2|^2 + 3|x_2|^2 + 4 = f(x_1, x_2)$
- ▶ Perfect approximation
- ▶ Finding good features is critical

Standard features: Gaussian basis functions



- ▶ The more features, the better the approximation
- ▶ ... but the more expensive the computation
- ▶ All the following algorithms use this structure
- ▶ In particular, we may use one kernel per known data point

Kernel Ridge Regression (KRR) = Kernel Regularised Least Squares (KRGLS)

- ▶ Define features with a kernel function $k(\mathbf{x}, \mathbf{x}_i)$ per point $\mathbf{x}_i$
- ▶ Define the Gram matrix as a kernel matrix:

$$\mathbf{K} = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & k(\mathbf{x}_1, \mathbf{x}_2) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ k(\mathbf{x}_2, \mathbf{x}_1) & k(\mathbf{x}_2, \mathbf{x}_2) & \cdots & k(\mathbf{x}_2, \mathbf{x}_N) \\ \vdots & \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & k(\mathbf{x}_N, \mathbf{x}_2) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{bmatrix}. \quad (7)$$

- ▶ If we had an infinity of data points, the linear approximation in feature space would become perfect
- ▶ **Intuition: the error is a function of the distance to data points**
- ▶ Computing the weights is done with RR using

$$\boldsymbol{\theta}^* = (\lambda \mathbf{I} + \mathbf{K})^{-1} \mathbf{y}, \quad (8)$$

- ▶ Note that $\mathbf{K}$ is symmetric
- ▶ The kernel matrix $\mathbf{K}$ grows with the number of points (kernel expansion)
- ▶ **The matrix inversion may become too expensive**
- ▶ Solution: finite set of features (RBFNs), incremental methods

Radial Basis Function Networks: definition and solution

- ▶ Radial Basis Functions versus Kernels (Gaussians

$\phi(\mathbf{x}, \boldsymbol{\theta}_e) = e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_e)^T \boldsymbol{\Sigma}_e^{-1}(\mathbf{x}-\boldsymbol{\mu}_e)}$ are both)

- ▶ We define a set of E basis functions (often Gaussian)

$$\hat{f}(\mathbf{x}) = \sum_{e=1}^E w_e \cdot \phi(\mathbf{x}, \boldsymbol{\theta}_e) \quad (9)$$

$$= \boldsymbol{\theta}^T \cdot \boldsymbol{\phi}(\mathbf{x}). \quad (10)$$

- ▶ We also define the *Gram matrix*

$$\mathbf{G} = \begin{bmatrix} \phi(\mathbf{x}_1, \boldsymbol{\theta}_1) & \phi(\mathbf{x}_1, \boldsymbol{\theta}_2) & \cdots & \phi(\mathbf{x}_1, \boldsymbol{\theta}_E) \\ \phi(\mathbf{x}_2, \boldsymbol{\theta}_1) & \phi(\mathbf{x}_2, \boldsymbol{\theta}_2) & \cdots & \phi(\mathbf{x}_2, \boldsymbol{\theta}_E) \\ \vdots & \vdots & \ddots & \vdots \\ \phi(\mathbf{x}_N, \boldsymbol{\theta}_1) & \phi(\mathbf{x}_N, \boldsymbol{\theta}_2) & \cdots & \phi(\mathbf{x}_N, \boldsymbol{\theta}_E) \end{bmatrix} \quad (11)$$

- ▶ and we get the least squares solution

$$\boldsymbol{\theta}^* = (\mathbf{G}^T \mathbf{G})^{-1} \mathbf{G}^T \mathbf{y}. \quad (12)$$

Least Square Projection Methods: summary of computations

► Linear case

$$\theta^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (LS) \quad (13)$$

$$\theta^* = (\lambda \mathbf{I} + \mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (RLS) \quad (14)$$

► Gram matrix case

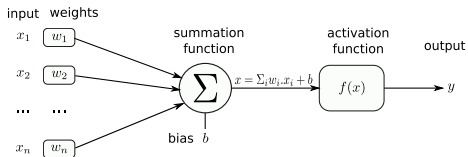
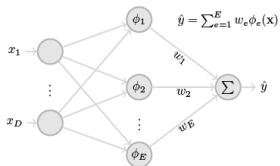
$$\theta^* = (\mathbf{G}^\top \mathbf{G})^{-1} \mathbf{G}^\top \mathbf{y} \quad (RBFN) \quad (15)$$

► Kernel matrix case

$$\theta^* = \mathbf{K}^{-1} \mathbf{y}, \quad (GPR) \quad (16)$$

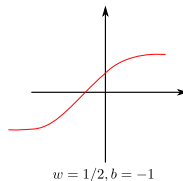
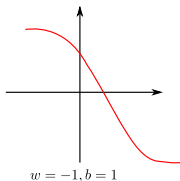
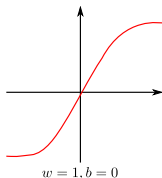
$$\theta^* = (\lambda \mathbf{I} + \mathbf{K})^{-1} \mathbf{y}. \quad (KRR) \quad (17)$$

The case of (feedforward) neural networks



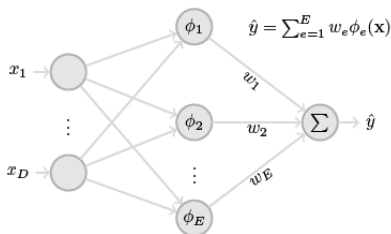
- ▶ The activation function is non local (sigmoid, ReLu, LeakyReLu...) vs Gaussians
- ▶ Weights of output layer: regression
- ▶ Weight of intermediate layer(s): tuning basis functions
- ▶ Shares the same structure as all basis function networks
- ▶ Sigmoids instead of Gaussians: better split of space in high dimensions

Regression with neural networks: discovering features



- ▶ The backprop algo tunes both kinds of weights
- ▶ Discovers the adequate features by itself
- ▶ Deep versus shallow: get more tunable features with less parameters
- ▶ Cannot be performed batch, see incremental methods (Classes 5, 6 and 7)

Regression with neural networks: variants

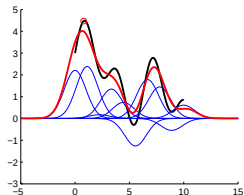
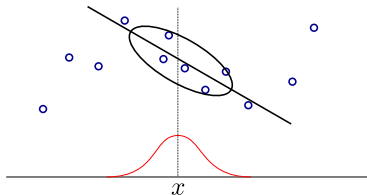


- ▶ If only the weights at the last layer are tuned, still defines a linear architecture (Extreme Learning Machine)
- ▶ Stochastic optimization of intermediate weights, linear regression on output weights?



Huang, G.-B., Zhou, H., Ding, X., & Zhang, R. (2012) Extreme learning machine for regression and multiclass classification. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 42(2):513–529

LWR versus RBFNs

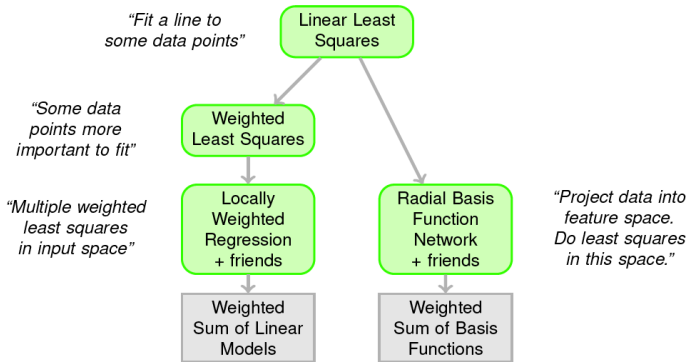


$$\hat{f}(\mathbf{x}) = \sum_{e=1}^E \phi(\mathbf{x}, \boldsymbol{\theta}_e) \cdot (b_e + \mathbf{a}_e^T \mathbf{x}) \quad (18)$$

$$\hat{f}(\mathbf{x}) = \sum_{e=1}^E \phi(\mathbf{x}, \boldsymbol{\theta}_e) \cdot w_e, \quad (19)$$

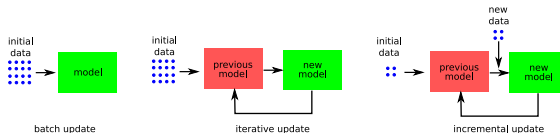
- ▶ Eq. (19) is a special case of (18) with $\mathbf{a}_e = \mathbf{0}$ and $b_e = w_e$.
- ▶ RBFNs: performs one LS computation in a projected space
- ▶ LWR: performs many LS computation in local domains

Wrap-up



► Image taken from Freck Stulp's IROS 2018 Tutorial

Batch, Iterative and Incremental learning

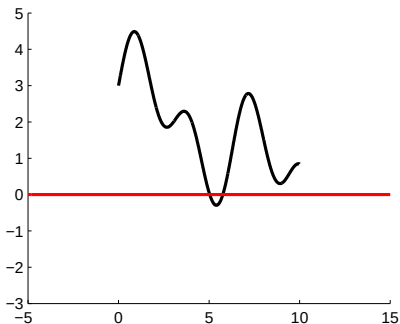


- ▶ So far, we have focused on batch regression
- ▶ Iterative methods: you improve a model through steps, using the same data at all steps
- ▶ Incremental methods: you improve a model through steps, with additional data at each step
- ▶ Incremental implies more than iterative
- ▶ Incremental approach to regression: reveal the batch data through steps

Limiting regression cost

- ▶ Solving $\theta^* = (\mathbf{G}^T \mathbf{G})^{-1} \mathbf{G}^T \mathbf{y}$ (or any variant) requires inverting $(\mathbf{G}^T \mathbf{G})$
- ▶ That is in $O(N^3)$ in the number of dimensions
- ▶ For large linear architectures or deep neural networks, this is too expensive
- ▶ Incremental solutions:
 - ▶ Complexity can be reduced to $O(N^2)$ by using the Sherman-Morrisson formula $\rightarrow$ incremental update of the inverse,
 - ▶ But sensitive to rounding errors.
 - ▶ Numerically more stable option: updating the Cholesky factor of the matrix using the QR algorithm.
 - ▶ In the linear case: Recursive Least Squares

Radial Basis Function Networks (Illustration)

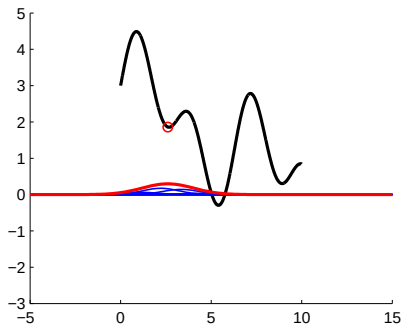


- Obtained using some incremental approach



Cybenko, G. (1989) Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4):303–314

Radial Basis Function Networks (Illustration)

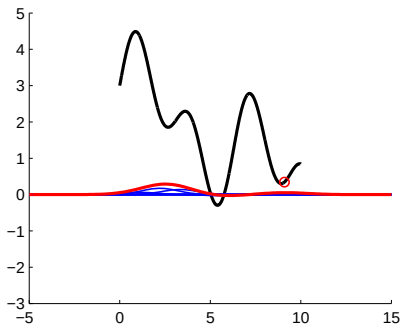


► Obtained using some incremental approach



Cybenko, G. (1989) Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4):303–314

Radial Basis Function Networks (Illustration)

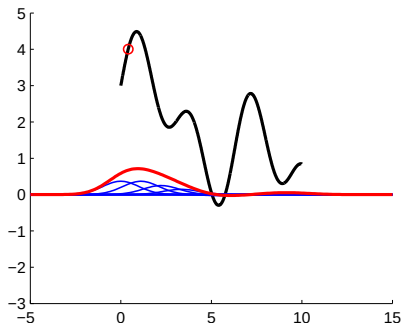


► Obtained using some incremental approach



Cybenko, G. (1989) Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4):303–314

Radial Basis Function Networks (Illustration)

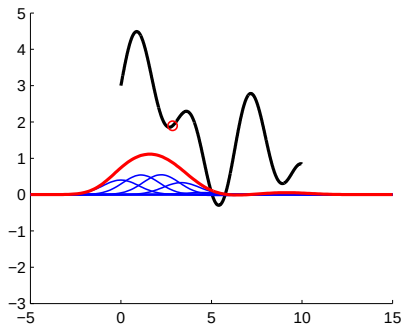


- Obtained using some incremental approach



Cybenko, G. (1989) Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4):303–314

Radial Basis Function Networks (Illustration)

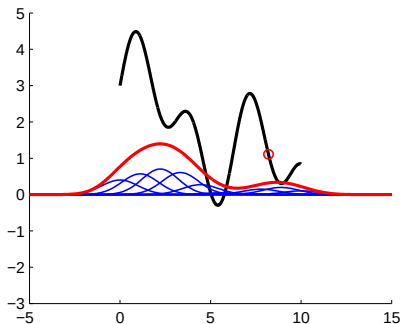


- Obtained using some incremental approach



Cybenko, G. (1989) Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4):303–314

Radial Basis Function Networks (Illustration)

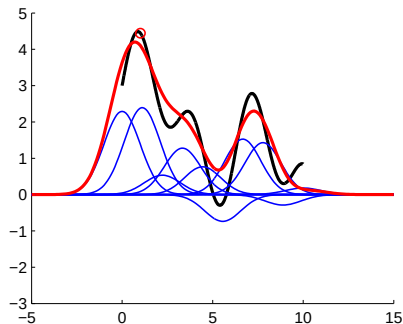


► Obtained using some incremental approach



Cybenko, G. (1989) Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4):303–314

Radial Basis Function Networks (Illustration)

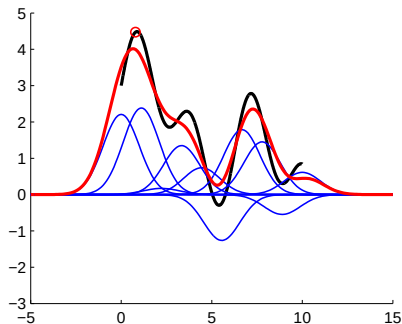


- Obtained using some incremental approach



Cybenko, G. (1989) Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4):303–314

Radial Basis Function Networks (Illustration)

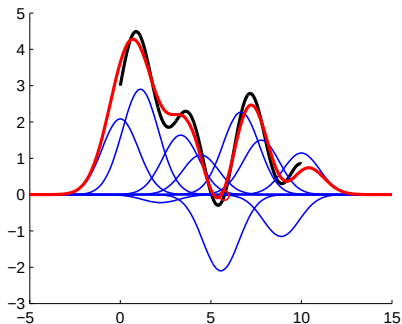


► Obtained using some incremental approach



Cybenko, G. (1989) Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4):303–314

Radial Basis Function Networks (Illustration)

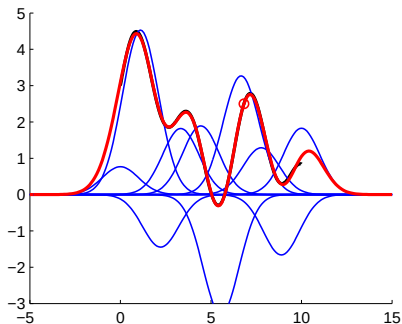


- Obtained using some incremental approach



Cybenko, G. (1989) Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4):303–314

Radial Basis Function Networks (Illustration)

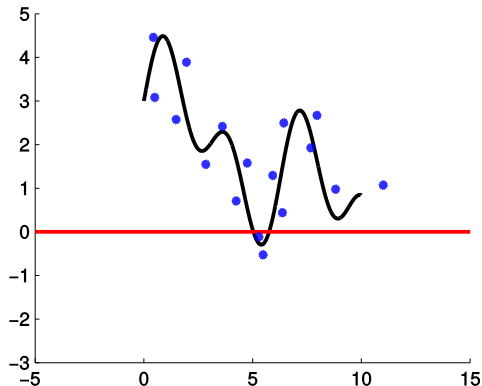


- Obtained using some incremental approach



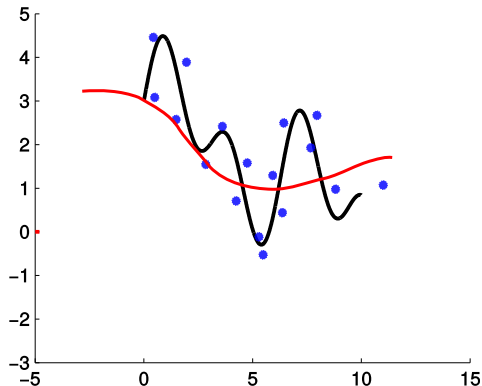
Cybenko, G. (1989) Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4):303–314

Risk of overfitting



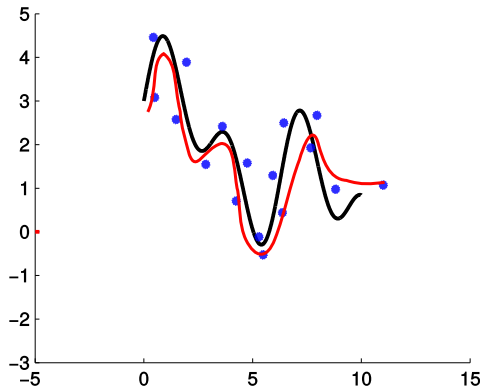
- ▶ In the previous simulation, measurements were noiseless
- ▶ What if we get noisy measurements?
- ▶ The risk is overfitting to data
- ▶ The question is when to stop

Risk of overfitting



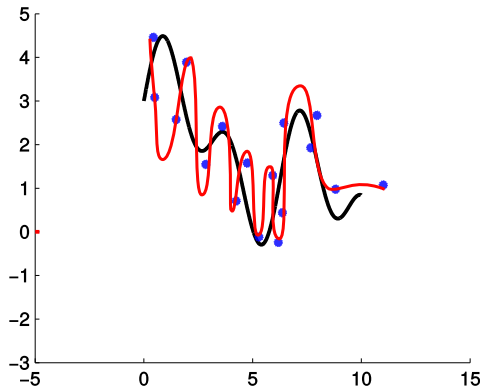
- ▶ In the previous simulation, measurements were noiseless
- ▶ What if we get noisy measurements?
- ▶ The risk is overfitting to data
- ▶ The question is when to stop

Risk of overfitting



- ▶ In the previous simulation, measurements were noiseless
- ▶ What if we get noisy measurements?
- ▶ The risk is overfitting to data
- ▶ The question is when to stop

Risk of overfitting



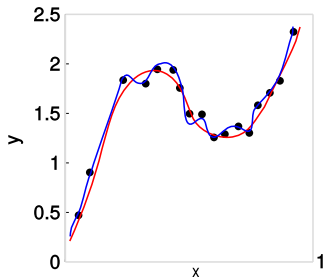
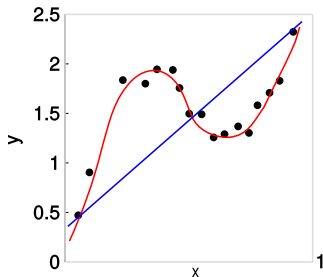
- ▶ In the previous simulation, measurements were noiseless
- ▶ What if we get noisy measurements?
- ▶ The risk is overfitting to data
- ▶ The question is when to stop

Fighting overfitting: Standard methodology



- ▶ Split the training set into training set and validation set
- ▶ Train on the training set
- ▶ Evaluate on the validation set
- ▶ When performance stops improving, stop iterating
- ▶ Should generalize well to the test set

Overfitting: influence of the model

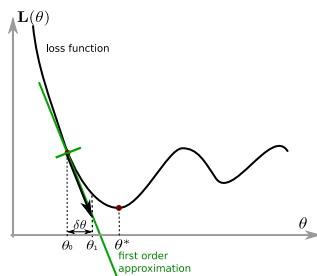
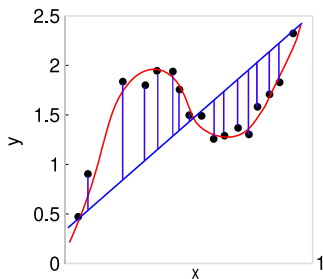


- ▶ A model with more parameters is more prone to overfitting
- ▶ If the model does not have enough free parameters:
 - ▶ The training error may not go down towards 0
 - ▶ The validation performance may stop improving
- ▶ → Select a rich enough model...
- ▶ ...and use the validation trick to decide when to stop
- ▶ (Deep) neural networks are particularly rich models

Gradient descent: Outline

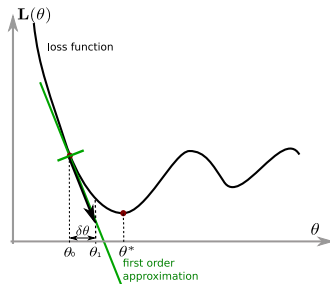
- ▶ How regression can be cast as a gradient descent problem
- ▶ What are the gradient descent properties independently from regression
- ▶ What is the difference between batch and stochastic gradient descent

Regression through gradient descent



- ▶ We want to minimize a regression error (left)
- ▶ Thus find the minimum of a **loss function** $L(\theta)$ (right)
- ▶ Instead of solving $\theta^* = \min_{\theta} L(\theta)$ (immediate, but expensive)
- ▶ Rather perform local steps using an iterative approach
- ▶ The iteration equation is $\theta_{i+1} \leftarrow \theta_i + \delta\theta_i$
- ▶ This is the essence of **gradient descent**

Gradient descent iteration

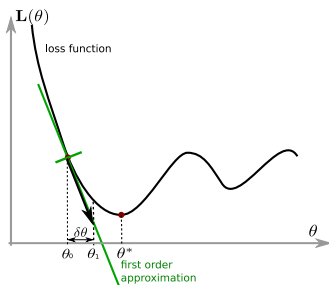


- Now we want to minimize $L(\theta_i + \delta\theta_i)$ over $\delta\theta_i$.
- How to choose $\delta\theta_i$?

First order approximation of the loss

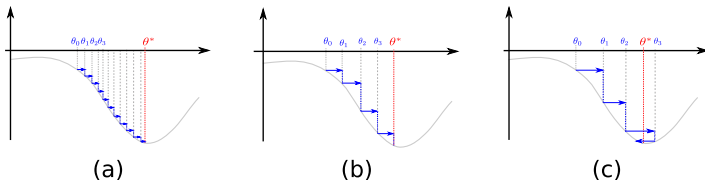
- ▶ The optimum on $\delta\theta$ is reached when $\frac{\partial \mathbf{L}(\theta + \delta\theta)}{\partial \delta\theta} = 0$
- ▶ $\mathbf{L}(\theta + \delta\theta)$ can be approximated at the first order as $\mathbf{L}(\theta) + \nabla_{\theta} \mathbf{L}(\theta)^T \delta\theta + \nu \delta\theta^T \delta\theta + \text{higher order terms}$
- ▶ $\frac{\partial \mathbf{L}(\theta + \delta\theta)}{\partial \delta\theta} \sim \nabla_{\theta} \mathbf{L}(\theta)^T \delta\theta + 2\nu \delta\theta$
- ▶ Thus $\delta\theta^* = -\frac{1}{2\nu} \nabla_{\theta} \mathbf{L}(\theta)$
- ▶ We rewrite it $\delta\theta^* = -\alpha \nabla_{\theta} \mathbf{L}(\theta)$
- ▶ And the iteration rule is $\theta_{i+1} \leftarrow \theta_i - \alpha_i \nabla_{\theta} \mathbf{L}(\theta)$
- ▶ Thus the steepest descent direction is given by the first order derivative

Gradient descent iteration



- ▶ The first order local derivative $\nabla_{\theta} L(\theta)|_{\theta=\theta_i}$ gives the right direction (move left or right)
- ▶ But minimizing the first order derivative is not lower bounded
- ▶ We need a step size α to determine how far to go

Tuning the step size

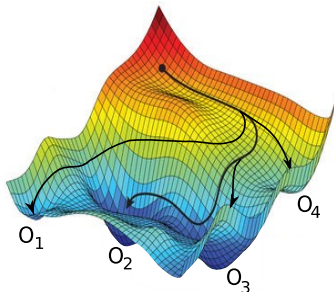


- ▶ (a) α_i are too small, (b) α_i are adequate, (c) α_i are too large
- ▶ If too small, too many steps. If too large, may miss a local optimum
- ▶ Line search: iterate to find the best step size (used e.g. in TRPO)



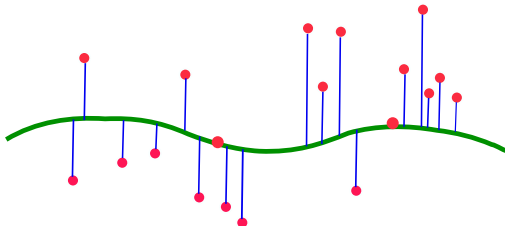
Schulman, J., Levine, S., Moritz, P., Jordan, M. I., & Abbeel, P. (2015) Trust region policy optimization. *CoRR*, [abs/1502.05477](https://arxiv.org/abs/1502.05477)

Local Optima



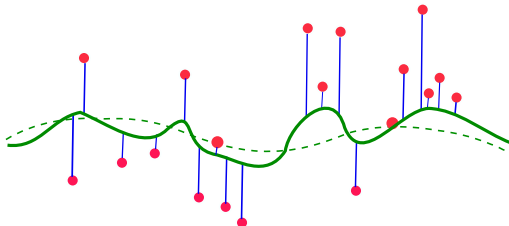
- ▶ Gradient descent is a local improvement approach where, at each step, we follow the steepest descent direction
- ▶ But we don't know where is the optimum we are targetting
- ▶ Unless the cost function is convex in the parameter space, gradient descent can end-up in different local optima depending on the starting point or noise
- ▶ Anything that basically goes down will end up somewhere low!

Batch regression through gradient descent



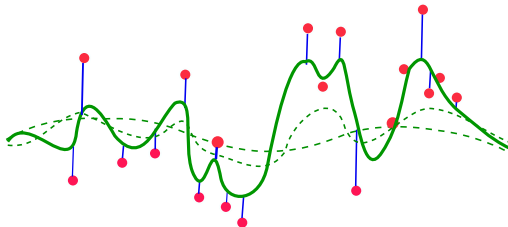
- ▶ The loss is the squared sum of all errors $L(\theta) = \|y - \hat{f}(X)\|^2$
- ▶ The weights of the regressor $\hat{f}$ are updated so as to decrease all errors on average
- ▶ The maths make it so that the regressor is corrected more where it makes larger errors

Batch regression through gradient descent



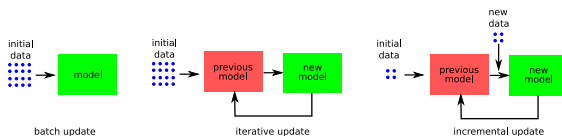
- ▶ The loss is the squared sum of all errors $L(\theta) = \|y - \hat{f}(X)\|^2$
- ▶ The weights of the regressor $\hat{f}$ are updated so as to decrease all errors on average
- ▶ The maths make it so that the regressor is corrected more where it makes larger errors

Batch regression through gradient descent



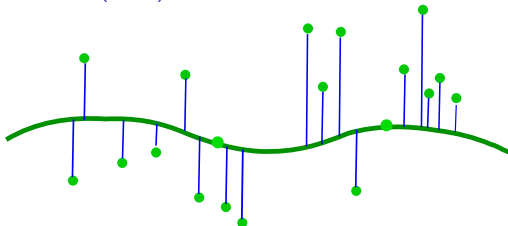
- ▶ The loss is the squared sum of all errors $L(\theta) = \|y - \hat{f}(X)\|^2$
- ▶ The weights of the regressor $\hat{f}$ are updated so as to decrease all errors on average
- ▶ The maths make it so that the regressor is corrected more where it makes larger errors

Reminder: Batch, Iterative, Incremental



- ▶ In regression, the cost function or loss is the regression error (e.g. $L(\theta) = \|\mathbf{y} - \hat{f}(\mathbf{X})\|^2$)
- ▶ The model function $\hat{f}(\mathbf{x})$ is known, so its derivative $\nabla_{\theta} \hat{f}(\mathbf{x})$ can be computed analytically
- ▶ The values $\mathbf{y}$ and $\mathbf{X}$ are known too
- ▶ Hence batch gradient descent is only iterative: no need for additional data
- ▶ Stochastic gradient descent is a specific case of incremental ($\mathbf{y}$ and $\mathbf{X}$ change for each mini-batch)
- ▶ The new datapoints are taken randomly from the batch

Stochastic gradient descent (SGD)

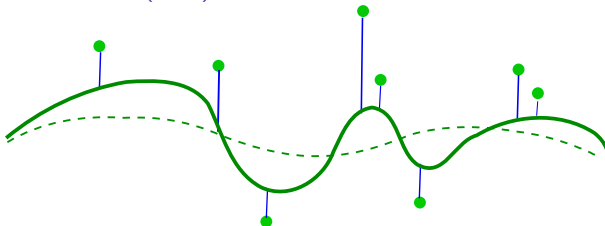


- ▶ Batch gradient descent is expensive when the batch is large
- ▶ We can decrease the cost by using several mini-batches (extra gain with parallel computations)
- ▶ In SGD, each mini-batch gives a different (inaccurate) estimate of the local gradient
- ▶ Combining several inaccurate estimates can help being accurate
- ▶ Actually, being inaccurate is no big deal (see Class 7)
- ▶ Note that SGD converges more slowly. Use it only for large batches.



Bottou, L. (2012) Stochastic gradient descent tricks. *Neural networks: Tricks of the trade*, pp. 421–436. Springer

Stochastic gradient descent (SGD)

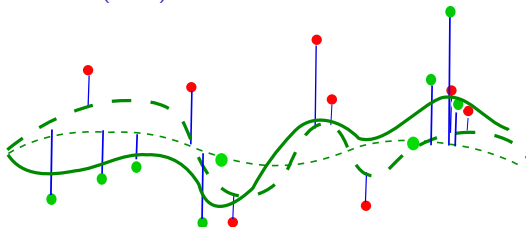


- ▶ Batch gradient descent is expensive when the batch is large
- ▶ We can decrease the cost by using several mini-batches (extra gain with parallel computations)
- ▶ In SGD, each mini-batch gives a different (inaccurate) estimate of the local gradient
- ▶ Combining several inaccurate estimates can help being accurate
- ▶ Actually, being inaccurate is no big deal (see Class 7)
- ▶ Note that SGD converges more slowly. Use it only for large batches.



Bottou, L. (2012) Stochastic gradient descent tricks. *Neural networks: Tricks of the trade*, pp. 421–436. Springer

Stochastic gradient descent (SGD)

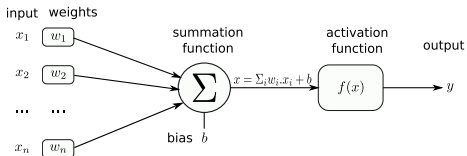
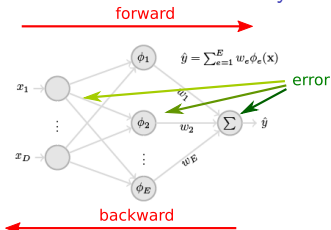


- ▶ Batch gradient descent is expensive when the batch is large
- ▶ We can decrease the cost by using several mini-batches (extra gain with parallel computations)
- ▶ In SGD, each mini-batch gives a different (inaccurate) estimate of the local gradient
- ▶ Combining several inaccurate estimates can help being accurate
- ▶ Actually, being inaccurate is no big deal (see Class 7)
- ▶ Note that SGD converges more slowly. Use it only for large batches.



Bottou, L. (2012) Stochastic gradient descent tricks. *Neural networks: Tricks of the trade*, pp. 421–436. Springer

Gradient descent over multilayer neural networks



- ▶ The parameters θ are the weights w_{ij} and biases b_i
- ▶ In $\nabla_{\theta}(\mathbf{L}(\theta))$, the errors at a layer depends on the errors at the next layers
- ▶ Gradient computation cannot be performed in a single step
- ▶ Compute the gradient with the gradient backpropagation algorithm (backprop)
- ▶ It is technical and a little tedious to code for each specific network structure
- ▶ TensorFlow, pytorch and their ancestors (theano, caffe...) are built to provide gradient backpropagation for any tensor structure



Nielsen, M. A. *Neural networks and deep learning*, volume 25. Determination press San Francisco, CA, 2015

<http://neuralnetworksanddeeplearning.com/chap2.html>

Creating an neural network in pytorch

```
class NeuralNetwork(nn.Module):  
  
    def __init__(self, l1, l2, l3, l4, out, learning_rate):  
        super(NeuralNetwork, self).__init__()  
        self.relu = nn.ReLU()  
        self.sigmoid = nn.Sigmoid()  
        self.fc1 = nn.Linear(l1, l2)  
        self.fc2 = nn.Linear(l2, l3)  
        self.fc3 = nn.Linear(l3, l4)  
        self.fc4 = nn.Linear(l4, out)  
        self.optimizer = th.optim.Adam(self.parameters(), lr=learning_rate)  
  
    def f(self, x):  
        input = th.from_numpy(x).float()  
        hidden1 = self.sigmoid(self.fc1(input))  
        hidden2 = self.sigmoid(self.fc2(hidden1))  
        hidden3 = self.sigmoid(self.fc3(hidden2))  
        output = rescale(self.sigmoid(self.fc4(hidden3)))  
        return output
```

- ▶ Adam does better than SGD
- ▶ `f()` is often called `forward()`
- ▶ Other functions not shown

Gradient descent in pytorch

- ▶ Computing the loss over a batch

```
for i in range(max_iter):  
    output = model.f(xt)  
    loss = func.mse_loss(output, yt)  
    model.update(loss)
```

- ▶ Applying gradient descent

```
def update(self, loss) -> None:  
    """  
    Apply a loss to a network using gradient backpropagation  
    :param loss: the applied loss  
    :return: nothing  
    """  
  
    self.optimizer.zero_grad()  
    loss.sum().backward()  
    self.optimizer.step()
```

- ▶ The backprop in one line!

Any question?



Send mail to: Olivier.Sigaud@upmc.fr



Atkeson, C.

Using locally weighted regression for robot learning.

In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, volume 2, pp. 958–963, Apr 1991.



Bishop, C. M.

Pattern recognition and machine learning.

Springer Berlin/Heidelberg, Germany, 2007.



Bottou, L.

Stochastic gradient descent tricks.

In *Neural networks: Tricks of the trade*, pp. 421–436. Springer, 2012.



Cleveland, W. S. and Devlin, S. J.

Locally weighted regression: an approach to regression analysis by local fitting.

Journal of the American statistical association, 83(403):596–610, 1988.



Cybenko, G.

Approximation by superpositions of a sigmoidal function.

Mathematics of Control, Signals, and Systems (MCSS), 2(4):303–314, 1989.



Gauss, C. F.

Theoria motus corporum coelestium in sectionibus conicis solem ambientium, volume 7.

Perthes et Besser, 1809.



Huang, G.-B., Zhou, H., Ding, X., and Zhang, R.

Extreme learning machine for regression and multiclass classification.

IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics), 42(2):513–529, 2012.



Legendre, A. M.

Nouvelles méthodes pour la détermination des orbites des comètes.

F. Didot, 1805.



Nielsen, M. A.

Neural networks and deep learning, volume 25.

Determination press San Francisco, CA, 2015.



Schulman, J., Levine, S., Abbeel, P., Jordan, M. I., and Moritz, P.

Trust region policy optimization.

In Bach, F. R. and Blei, D. M. (eds.), *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, volume 37 of *JMLR Workshop and Conference Proceedings*, pp. 1889–1897. JMLR.org, 2015.

URL <http://proceedings.mlr.press/v37/schulman15.html>.